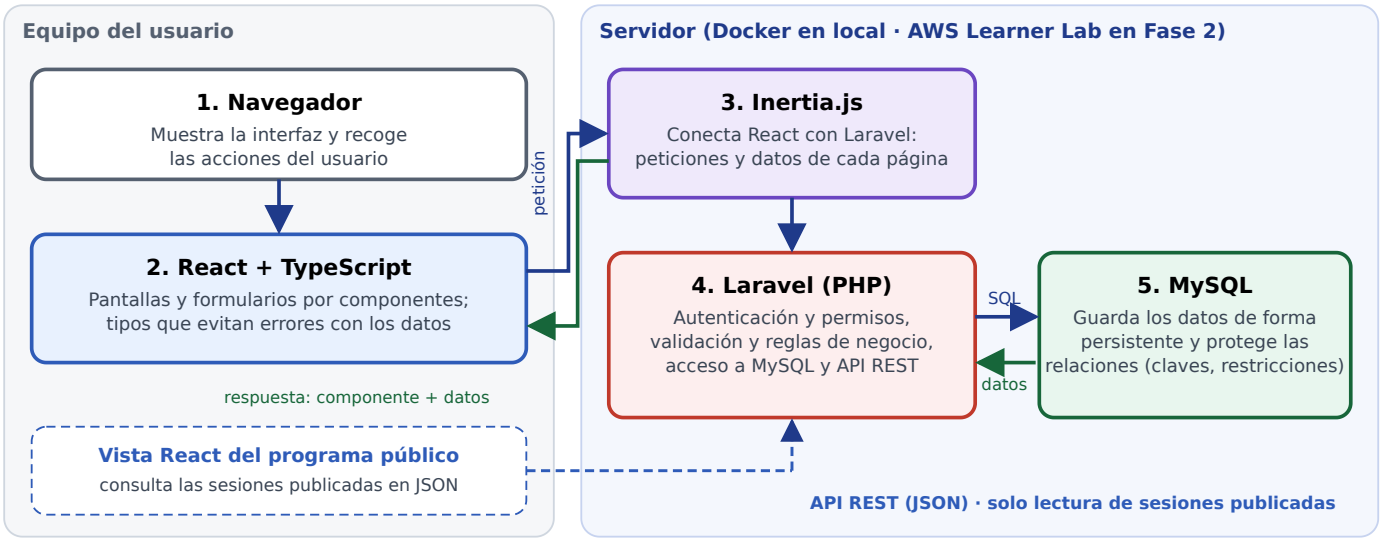
Tarea 2 - Arquitectura y datos

Caso TechFest. Pasos 7 y 8 de la tarea. Es una primera propuesta; seguramente cambie algo cuando empiece a programar.

1. Arquitectura



Qué hace cada parte

Parte	Qué hace
1. Navegador	Es donde el usuario ve las páginas, hace clic y rellena formularios. También guarda la cookie de la sesión iniciada.
2. React + TypeScript	Dibuja las pantallas a partir de componentes (la lista de sesiones, el formulario, los mensajes…). TypeScript sirve para indicar el tipo de cada dato, por ejemplo que el aforo es un número, y así se detectan errores antes de ejecutar.
3. Inertia.js	Une React con Laravel. Cuando el usuario cambia de página o envía un formulario, Inertia manda la petición a Laravel y le devuelve a React la página que tiene que mostrar con sus datos. Así no hace falta hacer una API aparte para el panel.
4. Laravel	Es el servidor. Comprueba quién es el usuario y si puede hacer lo que pide, valida los datos, guarda y lee en MySQL y ofrece la API REST del programa.
5. MySQL	Guarda todos los datos: cuentas, eventos, entradas, asistentes y sesiones. Con las claves primarias y foráneas se asegura de que las relaciones entre tablas sean correctas.

Cómo viaja una petición (crear una sesión)

1. El organizador rellena el formulario de nueva sesión (React) y pulsa «Guardar sesión».
2. **Inertia** envía los datos a Laravel junto con la cookie de sesión.
3. **Laravel** mira si el usuario ha iniciado sesión y si el evento es de su cuenta. Si no, devuelve acceso denegado.
4. Laravel **valida** los datos. Si algo está mal, devuelve los errores y React los enseña al lado de cada campo sin borrar lo que se había escrito.
5. Si todo está bien, Laravel **guarda** la sesión en **MySQL**. El evento y la cuenta los saca del usuario que ha iniciado sesión, no del formulario.
6. Laravel le dice a Inertia que vuelva al programa con el mensaje «Sesión creada correctamente».
7. **React** muestra la lista con la sesión nueva.

Para el **programa público** es parecido, pero la vista de React pide los datos a la API REST en JSON. Laravel solo devuelve las sesiones publicadas y solo sus datos públicos: título, inicio, fin, sala y aforo.

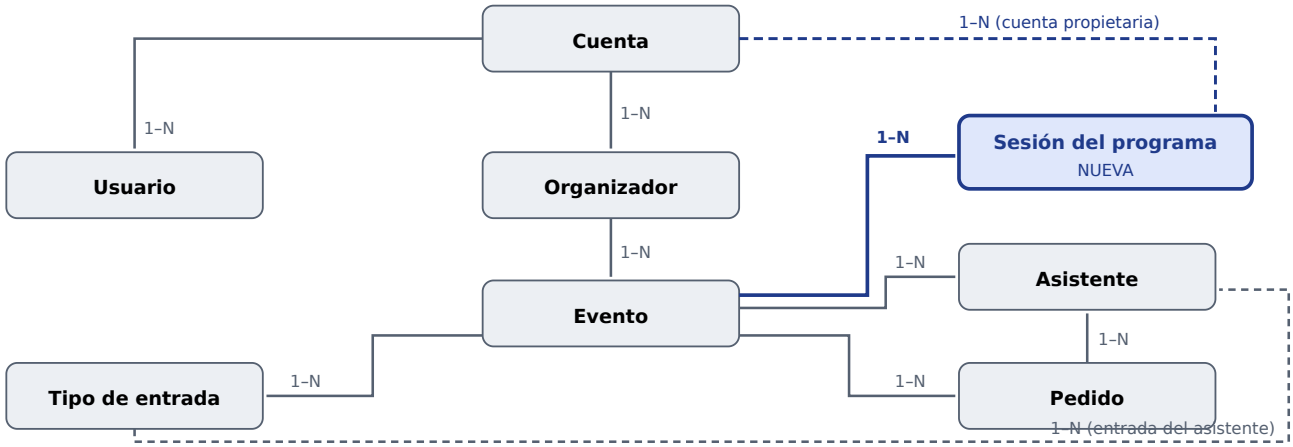
¿Actualizar Attendize o empezar un proyecto nuevo?

Opción	A favor	En contra
A. Actualizar Attendize poco a poco hasta una versión actual de Laravel	Se aprovecha todo el código que ya existe.	Hay que pasar por varias versiones de Laravel y actualizar muchas librerías antiguas (riesgo RS-01), y la interfaz seguiría en Blade y jQuery. Creo que no me daría tiempo.
B. Crear un proyecto Laravel nuevo con React, TypeScript e Inertia (la que he elegido)	Se usa desde el principio la tecnología que pide el curso y solo hago lo que entra en el alcance.	Tengo que volver a hacer las funciones que se conservan (RQ-09) y migrar los datos (RQ-10).

El enunciado permite hacerlo así si se explica qué parte nueva sustituye a cada función de Attendize y se migran los datos acordados. De todas formas, quiero confirmar esta decisión con el profesor en la validación. El Attendize original se queda sin tocar en la etiqueta `tarea-01-entrega`, para consultarlo y sacar los datos.

2. Datos

Entidades y relaciones



Una **cuenta** tiene usuarios y organizadores. Cada **evento** es de un organizador y de una cuenta, y tiene tipos de entrada, pedidos y asistentes. Cada asistente viene de un pedido y tiene un tipo de entrada.

La entidad nueva es la **sesión del programa**, que pertenece a un evento. También le guardo la cuenta para que sea más fácil comprobar los permisos.

Qué se conserva y qué es nuevo

Entidad	Qué pasa	Detalle
Cuentas, usuarios y organizadores	Se conservan	Los datos de cada uno y a qué cuenta pertenecen. Las contraseñas ya están cifradas y se copian así, nunca en texto plano.
Eventos	Se conservan	Título, descripción, fechas, si está publicado, organizador y cuenta.
Tipos de entrada	Se conservan	Nombre, precio (ficticio), cantidad y evento.
Pedidos y asistentes	Se conservan	Con qué evento y entrada están relacionados, su estado y los datos personales (ficticios) que hagan falta.
Sesiones del programa	**Es nueva**	No existe en Attendize (H1). Los datos de prueba se crearán con un *seeder* y desde la propia aplicación.
El resto de Attendize	No se migra	Pagos, códigos de descuento, afiliados, encuestas, imágenes, widgets y mensajes. Están fuera del alcance.

Correspondencia entre datos antiguos y nuevos

Ojo: los nombres de las columnas de Attendize los tengo que comprobar con `DESCRIBE` en la base de datos de la Tarea 1 antes de hacer la migración.

Attendize v2.8.0	Aplicación nueva	Qué cambia
`accounts`: id, first_name, last_name, email	`accounts`: los mismos campos	Se mantiene el mismo `id`. El email se guarda en minúsculas y sin espacios.
`users`: id, account_id, first_name, last_name, email, password	`users`: id, account_id, name, email, password	`name` junta nombre y apellidos. La contraseña cifrada se copia tal cual; hay que comprobar que la versión nueva la acepta. El email no se puede repetir.
`organisers`: id, account_id, name, email	`organisers`: los mismos campos	No cambia.
`events`: id, account_id, organiser_id, title, description, start_date, end_date, is_live	`events`: los mismos, pero `is_published` en vez de `is_live`	Cambio `is_live` por `is_published` porque se entiende mejor. Significa lo mismo.
`tickets`: id, event_id, title, price, quantity_available, quantity_sold	`tickets`: los mismos campos	Si `quantity_available` está vacío significa «sin límite» y se deja igual.
`orders`: id, event_id, order_status_id, first_name, last_name, email, order_reference	`orders`: los mismos campos	Como no hay pagos reales, solo se guardan el estado y la referencia.
`attendees`: id, order_id, event_id, ticket_id, first_name, last_name, email, has_arrived, is_cancelled	`attendees`: los mismos campos	No cambia.
No existe	`event_sessions`: id, account_id, event_id, title, starts_at, ends_at, room, capacity, is_published, fechas de creación y modificación	Tabla nueva. No la llamo `sessions` porque Laravel ya usa una tabla con ese nombre para las sesiones de los usuarios.

Las fechas de creación, modificación y borrado (`created_at`, `updated_at`, `deleted_at`) se copian tal cual en todas las tablas.

Validaciones

Dato	Regla (la comprueba Laravel)
Título de la sesión	Obligatorio, máximo 150 caracteres.
Inicio y fin	Obligatorios y con fecha y hora válidas. El fin tiene que ser después del inicio.
Sala	Obligatoria, máximo 150 caracteres.
Aforo	Obligatorio, número entero mayor que 0.
Publicada	Sí o no. Por defecto, no publicada.
Evento y cuenta	No vienen del formulario: los saca el servidor del evento del usuario que ha iniciado sesión.
Por decidir	Si la sesión tiene que estar dentro de las fechas del evento y si el aforo tiene un máximo (dudas 2 y 3 de la propuesta).
Eventos	Título obligatorio y la fecha de fin después de la de inicio, igual que en Attendize.

Reglas para la migración

Caso	Qué se hace

Antes de empezar	Hacer una copia completa de la base de datos de Attendize con `mysqldump` y comprobar que se puede restaurar.
Faltan datos	Si falta un dato obligatorio (por ejemplo, un evento sin fecha de fin), no me lo invento: lo apunto en una lista de incidencias y decido qué hacer con él. Si falta un dato opcional, se deja vacío.
Duplicados	Si hay dos usuarios con el mismo email, se queda el más antiguo y el otro se apunta para revisarlo a mano.
Registros sueltos	Por ejemplo, un asistente de un pedido que no existe: no se migra y se apunta en la lista de incidencias.
Registros borrados	Los que tienen `deleted_at` se migran igual para no romper relaciones, pero la aplicación no los enseña.
Repetir la migración	Como se mantienen los `id` originales, si un registro ya existe se actualiza en vez de crear otro. Así no se duplica nada (CA-10.2).
Comprobar	Comparar cuántos registros hay en cada tabla antes y después, algunas relaciones (eventos por organizador, asistentes por evento) y unos cuantos registros a mano (CA-10.1).